



Bienvenidos a la tercera presentación dedicada a bases de datos distribuidas en donde vamos a ver las estrategias que se pueden usar en el diseño de bases de datos distribuidas.

Bases de datos distribuidas

- ¿En qué consiste el diseño de BD distribuidas?
- Estrategias de distribución
- Diseño de BD distribuidas

EIMT.UOC.EDU

Esta presentación se divide en tres secciones.

En la primera veremos en qué consiste el diseño de bases de datos distribuidas.

En la segunda sección explicaremos las estrategias que podemos usar en el diseño de bases de datos distribuidas.

Finalmente, en la tercera, estudiaremos cuáles de dichas estrategias están disponibles, tanto en el caso de bases de datos relacionales, como de bases de datos NoSQL.

Bases de datos distribuidas

- ¿En qué consiste el diseño de BD distribuidas?
- Estrategias de distribución
- Diseño de BD distribuidas

Empezamos, pues, explicando en qué consiste el diseño de bases de datos distribuidas y cuáles son los objetivos que se pretenden conseguir.

¿En qué consiste el diseño de BD distribuidas?

- El diseño de BDD toma decisiones acerca de cómo fragmentar la BD, qué fragmentos se deben replicar, y dónde almacenar los fragmentos y réplicas diseñados.
- Para ello es necesario analizar las necesidades de los usuarios/aplicaciones:
 - Qué parte de la BD necesitan
 - Qué operaciones realizan sobre la BD
 - Cuál es su punto de acceso al sistema
- Los objetivos a conseguir son:
 - Disminuir los costes de transmisión de datos
 - Repartir la carga de trabajo
 - Aumentar la eficiencia y promover la disponibilidad de los datos

EIMT.UOC.EDU

El diseño de bases de datos distribuidas consiste en tomar decisiones en relación a una serie de aspectos. Antes de entrar en ellos, recordemos que existen diferentes situaciones que pueden conducir a la existencia de una base de datos distribuida. Una es que se trate de una decisión tomada por el equipo que diseña la base de datos de acuerdo con la organización o empresa que la utilizará.

En este caso, a las estrategias de diseño seguidas en una base de datos centralizada, se deben añadir otras que ayuden a tomar decisiones relativas a cómo dividir la base de datos en diferentes fragmentos, si es o no necesario replicar algunos de esos fragmentos (o todos) y dónde se van almacenar los fragmentos (y sus réplicas, si las hubiera).

Para poder realizar un buen diseño de la base de datos distribuida es necesario analizar cuidadosamente las necesidades de las aplicaciones (y en consecuencia de los usuarios). En concreto, hay que considerar qué parte de la base de datos necesitan acceder, qué funcionalidades hay que suministrar (es decir, qué operaciones se realizan sobre los datos y cómo se accede a éstos) y cuál es el punto de acceso de los usuarios al sistema (es decir, la idea es acercar los datos a dónde éstos se necesitan).

Los objetivos que se esperan conseguir son principalmente tres. En primer lugar, disminuir los costes de transmisión de datos a través de la red. En segundo lugar, repartir la carga de trabajo de forma uniforme entre los diferentes nodos. Y en tercer lugar, y como consecuencia de las dos anteriores, aumentar la eficiencia en el procesamiento de las peticiones formuladas, y garantizar la disponibilidad de los datos.

Idealmente, la existencia de fragmentos y sus réplicas, así como el lugar donde estos elementos estén almacenados, debe ser transparente a todos los usuarios. Esta noción se conoce como transparencia de distribución.

Bases de datos distribuidas

- ¿En qué consiste el diseño de BD distribuidas?
- Estrategias de distribución
- Diseño de BD distribuidas

EIMT.UOC.EDU

A continuación vamos a ver en qué consisten la fragmentación y replicación de los datos. Éstas son las estrategias que podemos utilizar en el diseño de bases de datos distribuidas. Tras esto analizaremos cuáles son los principales beneficios y dificultades que se derivan de su uso. Finalmente, estudiaremos los diferentes tipos de fragmentación, y se ejemplificarán en el caso particular de bases de datos relacionales. Debemos tener en cuenta que la fragmentación y replicación son estrategias inicialmente pensadas para el ámbito de bases de datos relacionales. A pesar de ello, los conceptos teóricos que subyacen y las conclusiones alcanzadas son también válidas para las bases de datos NoSQL.

Estrategias de distribución

- **Fragmentación:** los datos/esquema se dividen en subconjuntos. Existen tres tipos de fragmentación:
 - **Horizontal:** toma como unidad de distribución conjuntos de datos relacionados.
 - **Vertical:** usa subconjuntos del esquema (y los datos asociados) como unidad de distribución, por ejemplo, grupos de atributos.
 - **Híbrida:** mezcla de las dos anteriores.
- **Replicación:** consiste en reproducir fragmentos del esquema (y de los datos que éstos contienen).

EIMT.UOC.EDU

La fragmentación se basa en la idea de que las aplicaciones (y por lo tanto los usuarios), en general, no necesitan acceder a toda la base de datos, sino que están interesados en acceder a subconjuntos de la misma. Por lo tanto, el objetivo de la fragmentación es encontrar la unidad de acceso a los datos más apropiada para cada dominio de aplicación. Dicha unidad de acceso (que se denomina fragmento) se convierte en la unidad de distribución entre los diferentes nodos que forman la base de datos distribuida. Existen tres tipos de fragmentación: la horizontal, la vertical y la híbrida.

La fragmentación horizontal toma conjuntos de datos relacionados como unidad de distribución, de tal manera que objetos individuales o subconjuntos de objetos que comparten semántica (es decir, objetos que pertenecen a una misma clase) pasan a ser la unidad de distribución entre los diferentes nodos. Por su parte, la fragmentación vertical consiste en usar subconjuntos del esquema, en general, grupos de atributos de los objetos (y los datos asociados a ellos) como unidad de distribución. Finalmente, la fragmentación híbrida combina la fragmentación horizontal y vertical.

La segunda estrategia de distribución es la replicación. Ésta consiste en almacenar todos o una parte de los fragmentos diseñados (y por lo tanto, los datos que éstos contienen) en más de un nodo.

Las estrategias de fragmentación y replicación son ortogonales entre sí. En consecuencia, podemos optar por usar únicamente fragmentación (la base de datos se divide en fragmentos que se distribuyen entre los diferentes nodos, pero ningún fragmento estará replicado), podemos no fragmentar la base de datos pero sí replicarla (si se replica de forma completa estaríamos hablando de técnicas de *mirroring* de bases de datos) o bien podemos utilizar una combinación de ambas estrategias. Esta última acostumbra a ser la elección habitual en el caso de una base de datos distribuida.

Estrategias de distribución

Fragmentación	Replicación
+ Permite acercar los datos allá donde se necesitan (<i>data locality</i>).	+ Permite acercar los datos allá donde se necesitan (<i>data locality</i>).
+ Incrementa el nivel de concurrencia y facilita paralelismo.	+ Incrementa la disponibilidad de los datos.
- El rendimiento empeora cuando las operaciones necesitan recuperar múltiples fragmentos almacenados en diferentes nodos.	+ Permite mejorar la eficiencia de las operaciones de consulta (<i>read only</i>).
- El control semántico de los datos se puede complicar (restricciones de integridad).	- La inserción de objetos y su modificación es menos eficiente.
	- Compromete la consistencia (todas las réplicas de un mismo objeto deben ser idénticas).

EIMT.UOC.EDU

A continuación resumimos los principales beneficios y dificultades que se derivan del uso de las estrategias de distribución que acabamos de presentar.

La fragmentación permite acercar los datos allá donde se necesitan. Esta característica se conoce como localidad de los datos (en inglés *data locality*). Si un mismo fragmento debe ser accedido desde diferentes ubicaciones, se puede replicar. Por lo tanto, la replicación ayuda a maximizar la localidad de los datos.

Asimismo la fragmentación permite que más operaciones se ejecuten al mismo tiempo y en paralelo, de tal manera que se incrementa el rendimiento global del sistema. Este aspecto es de gran importancia en bases de datos altamente distribuidas que almacenan grandes volúmenes de datos. La existencia de réplicas, además, incrementa la disponibilidad de los datos ante situaciones de avería (si un nodo no está disponible o accesible, los datos se pueden recuperar de otros nodos). La replicación también permite mejorar significativamente la eficiencia de las operaciones de consulta.

A pesar de estos beneficios, la fragmentación puede conducir a un peor rendimiento cuando hay que acceder a fragmentos que están almacenados en diferentes nodos. Por ejemplo, esta situación se puede dar cuando hay requerimientos contradictorios que hacen imposible separar los datos en fragmentos mutuamente excluyentes. En este sentido, la situación ideal sería que cada usuario sólo necesitase acceder a un nodo para recuperar los datos que necesita, y que el acceso de los usuarios se repartiese de forma uniforme entre todos los nodos, porque esto permitiría equilibrar la carga de trabajo.

Los problemas de rendimiento descritos se podrían ver agravados si existen reglas de integridad que implican a fragmentos diferentes almacenados en distintos nodos.

Finalmente, la replicación complica las operaciones de inserción, borrado y modificación de los datos. En concreto, la inserción y modificación son especialmente problemáticas. La inserción de nuevos objetos, incluye también la inserción de sus réplicas (si las hubiera). En el caso de modificación, es necesario garantizar que todas las réplicas de un mismo objeto contengan los mismos valores. Si esto no es así, se está comprometiendo la consistencia.

Existen diferentes políticas para garantizar que las réplicas de un mismo objeto sean idénticas. Estas políticas surgen de considerar dos dimensiones. La primera tiene que ver con el hecho de si la consistencia se mantiene de forma síncrona o asíncrona (es decir, si los cambios realizados sobre una réplica se propagan de forma inmediata o diferida al resto). La segunda dimensión se relaciona con la existencia de diferentes tipos de réplicas (una copia primaria donde se realizan los cambios que luego serán propagados al resto) o no (todas las réplicas son iguales). Estas dimensiones se pueden combinar dando lugar a diferentes políticas como sería el caso de la replicación *master-slave* y la replicación P2P. Estas políticas serán objeto de estudio en presentaciones posteriores.

Fragmentación horizontal

- Especialmente adecuada para organizaciones geográficamente dispersas que principalmente acceden a los datos que guardan localmente (maximiza la *data locality*).
- Los filtros (o condiciones) aplicados en las operaciones que formulan los usuarios/aplicaciones pueden servir para decidir qué fragmentos crear.

La relación Cliente se divide en tres fragmentos en función del valor del atributo paísCliente.

Cliente					
idCliente	nombreCliente	paísCliente	totalFacturado	numProyectos	idComercial
c1	FX	Francia	30000	3	com1
c2	FW	Francia	100000	5	com2
c3	FT	Francia	50000	1	com1

idCliente	nombreCliente	paísCliente	totalFacturado	numProyectos	idComercial
c4	UX	USA	6000	1	com10
c5	UW	USA	12000	2	com23

idCliente	nombreCliente	paísCliente	totalFacturado	numProyectos	idComercial
c6	EX	España	100000	4	com1
c7	EW	España	40000	2	com2
c8	ET	España	25000	1	com1
c9	EZ	España	12000	8	com1

EIMT.UOC.EDU

La fragmentación horizontal toma como unidad de distribución objetos individuales o subconjuntos de objetos. Estos subconjuntos de objetos comparten semántica (en definitiva, son instancias de una misma clase). En general, los subconjuntos se establecen en función del valor de un atributo o grupo de atributos del objeto.

Si pensamos en una base de datos distribuida relacional, la fragmentación horizontal consiste en dividir una relación en relaciones más pequeñas que contienen subconjuntos de filas de la relación original. Cada una de estas relaciones más pequeñas constituye un fragmento. Por ejemplo, dada una relación de clientes, podríamos fragmentar dicha relación en función de su país de procedencia (que es un atributo de la relación). Cada fragmento contiene todos los atributos del cliente.

Diferentes fragmentos que estén relacionados se pueden almacenar en un mismo nodo. Esto puede ser especialmente interesante, por ejemplo, para organizaciones geográficamente dispersas que principalmente acceden a los datos que guardan localmente.

Para decidir qué fragmentos se deben crear se suelen analizar las condiciones usadas por los usuarios en las peticiones que formulan, o el patrón que siguen en el acceso a los datos.

Fragmentación vertical

- Para el diseño de fragmentos es necesario analizar la afinidad entre atributos (qué atributos se recuperan de forma conjunta en las operaciones de consulta).
- Pueden complicar las operaciones de inserción y modificación, y comprometer la consistencia de los datos.
- Ayuda a mejorar el ratio de datos útiles recuperados en operaciones de consulta (sólo se leen los atributos relevantes).
- Especialmente útil en sistemas decisionales

La relación Cliente se divide en dos fragmentos con grupos de atributos diferentes.

Cliente						
idCliente	nombreCliente	paísCliente	idCliente	totalFacturado	numProyectos	idComercial
c1	FX	Francia	c1	30000	3	com1
c2	FW	Francia	c2	100000	5	com2
c3	FT	Francia	c3	50000	1	com1
c4	UX	USA	c4	6000	1	com10
c5	UW	USA	c5	12000	2	com23
c6	EX	España	c6	100000	4	com1
c7	EW	España	c7	40000	2	com2
c8	ET	España	c8	25000	1	com1
c9	EZ	España	c9	12000	8	com1

EIMT.UOC.EDU

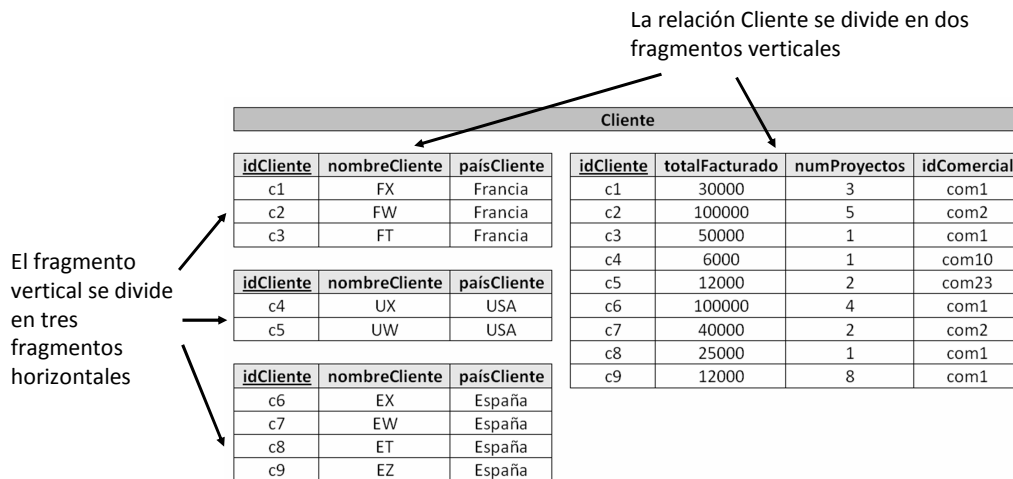
Por su parte, la fragmentación vertical consiste en usar subconjuntos del esquema, en general, grupos de atributos de los objetos (y los datos asociados a ellos) como unidad de distribución. Tradicionalmente la fragmentación vertical ha sido poco utilizada, por diversas razones. En primer lugar, la decisión de qué atributos agrupar en cada fragmento requiere analizar su afinidad y esto puede ser muy complejo. En segundo lugar, la fragmentación vertical puede introducir complicaciones en la inserción de nuevos objetos y en la modificación de los existentes. Pensemos que los atributos de los objetos (y en consecuencia, los propios objetos) pueden haber quedado divididos en diferentes fragmentos que, además, pueden estar almacenados en diferentes nodos.

A pesar de las dificultades reseñadas, con la irrupción de los sistemas decisionales (en donde a los usuarios sólo necesitan consultar datos), esta estrategia de fragmentación emerge como una alternativa poderosa para procesar de forma eficiente operaciones que implican la lectura de grupos de atributos de todos o una parte de los objetos almacenados. Los almacenes de columnas (*column stores*) constituyen el ejemplo más claro de sistema basado en los principios de la fragmentación vertical.

La figura muestra un ejemplo de fragmentación vertical sobre una base de datos distribuida relacional. En este caso, se han diseñado dos fragmentos para la relación de clientes. El primero contiene el identificador del cliente, su nombre y país de procedencia, mientras que el segundo contiene el identificador del cliente y los datos sobre el total facturado, el número de proyectos contratados, y el comercial que cada cliente tiene asignado.

Fragmentación híbrida

- Combina fragmentación horizontal y vertical.



EIMT.UOC.EDU

Finalmente, la fragmentación híbrida combina las estrategias de fragmentación vertical y horizontal, tal y como muestra el ejemplo.

En este caso, y tras haber fragmentado verticalmente en dos fragmentos la relación de clientes, se aplicó fragmentación horizontal (en función del país de procedencia del cliente) sobre el primero de dichos fragmentos.

Bases de datos distribuidas

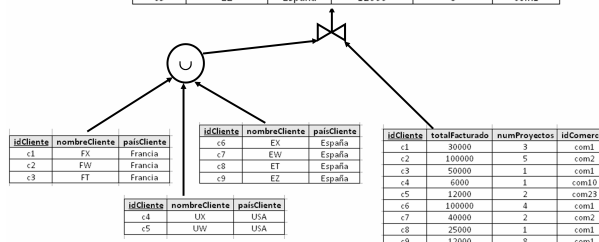
- ¿En qué consiste el diseño de BD distribuidas?
- Estrategias de distribución
- Diseño de BD distribuidas

En esta sección vamos a explicar las particularidades asociadas a las estrategias de distribución que acabamos de discutir cuando se aplican, respectivamente, en bases de datos relacionales y en bases de datos NoSQL. También situaremos el uso de dichas estrategias dentro del proceso general de diseño de bases de datos.

Diseño de BD distribuidas: BD relacionales

- Las BD relacionales soportan los diferentes tipos de fragmentación y también replicación.
- El esquema de fragmentación tiene que cumplir tres propiedades: completitud, disyunción y reconstrucción.

Cliente					
idCliente	nombreCliente	paisCliente	totalFacturado	numProyectos	idComercial
c1	FX	Francia	30000	3	com1
c2	FW	Francia	100000	5	com2
c3	FT	Francia	50000	1	com1
c4	UX	USA	6000	1	com10
c5	UW	USA	12000	2	com23
c6	EX	España	100000	4	com1
c7	EW	España	40000	2	com2
c8	ET	España	25000	1	com1
c9	EZ	España	12000	8	com1



EIMT.UOC.EDU

Como hemos ejemplificado, las bases de datos relacionales soportan los diferentes tipos de fragmentación y también replicación.

Los criterios de asignación de datos a fragmentos, las réplicas que tienen que existir de cada fragmento, así como la distribución de dichos elementos entre los nodos que conforman la base de datos distribuida son decisiones que toma el equipo responsable del diseño de la base de datos.

Finalmente, el esquema de fragmentación debe cumplir las propiedades de completitud, disyunción y reconstrucción. La completitud garantiza que no se pierdan datos como consecuencia del proceso de fragmentación. Por su parte, la disyunción indica que cada elemento de datos (fila o columna) sólo puede estar en un fragmento, a excepción de aquellas relaciones que hayan sido fragmentadas verticalmente. En este caso, la clave primaria de la relación deberá aparecer en todos los fragmentos. Finalmente, la reconstrucción significa que la relación original se tiene que poder regenerar a partir de sus fragmentos. La presencia de la clave primaria en todas las relaciones que hayan sido fragmentadas verticalmente garantiza que la relación original pueda ser reconstruida.

En esencia, las propiedades previas garantizan, por una parte, que el proceso de fragmentación preserve la semántica de la base de datos y de los datos almacenados en ella (completitud y reconstrucción). Por otra parte, garantiza que se eviten redundancias, es decir, repeticiones de los datos que serían evitables (disyunción). En relación a este último punto, recordemos que las bases de datos relacionales promulgan la normalización de los datos. La disyunción no es contradictoria con la existencia de réplicas. Si bien la replicación de fragmentos introduce redundancias, éstas serán controladas por el gestor de la base de datos.

A modo de ejemplo, y tal como muestra la figura, el esquema de fragmentación híbrida que hemos creado verifica las propiedades de completitud (todos los datos contenidos en la relación de clientes están en algún fragmento), disyunción (cada elemento de datos está sólo en un fragmento, a excepción de la clave primaria de la relación clientes) y reconstrucción (la relación de clientes se puede reconstruir haciendo, en primera instancia, la unión de los 3 fragmentos mostrados en la parte izquierda de la figura, y a continuación la combinación (*join*) con el fragmento de la derecha. En el ejemplo se asume que los países de procedencia de los clientes únicamente pueden ser Francia, USA o España.

Diseño de BD distribuidas: BD NoSQL

- Las BD NoSQL basadas en modelos de agregación permiten la replicación de datos y promueven la fragmentación horizontal (que se conoce bajo la denominación de *sharding*).
- El agregado constituye la unidad mínima a efectos de distribución de datos.
- En las BD NoSQL, en ocasiones, las decisiones sobre en qué fragmentos se almacenan los datos y dónde se almacenan los fragmentos son realizadas automáticamente por el gestor de la BD (*auto-sharding*).
- El esquema de fragmentación debe cumplir las propiedades de completitud y reconstrucción.

EIMT.UOC.EDU

A continuación presentamos las estrategias de distribución usadas por las bases de datos NoSQL. Destacar que se omiten las de modelos en grafo, dado que éstas están primordialmente pensadas para ser centralizadas.

Las bases de datos NoSQL basadas en modelos de agregación permiten la replicación (en algunos casos masiva) de los datos y promueven principalmente la fragmentación horizontal. En el contexto de NoSQL, la fragmentación horizontal recibe el nombre de *sharding*.

Recordemos que en los modelos de agregación, el agregado es la unidad a efectos de acceso (y manipulación) y de control de concurrencia y consistencia. Por estos motivos, también constituye la unidad mínima a efectos de distribución de datos. Podemos agrupar agregados de un mismo tipo en fragmentos. Cada uno de estos fragmentos recibe el nombre de *shard*. Cada fragmento (o *shard*) estará almacenado, al menos, en un nodo de la base de datos distribuida.

Mientras que en las bases de datos relacionales la asignación de datos a fragmentos y la distribución de fragmentos entre los diferentes nodos constituyen decisiones que toman el equipo responsable del diseño de la base de datos, en NoSQL esta decisión se puede delegar al gestor de la base de datos. Esto se conoce como *auto-sharding*.

El esquema de fragmentación debe preservar la semántica de la base de datos y de los datos almacenados en ella. En definitiva, el esquema de fragmentación debe garantizar las propiedades de completitud y reconstrucción. La disyunción no es necesaria, dado que las bases de datos NoSQL se basan en la desnormalización de los datos.

Diseño de BD distribuidas: BD NoSQL

- Las BD clave-valor distribuyen los agregados, en general, apoyándose en el uso de técnicas de *hash* (por ejemplo, *consistent hashing*).
- Las BD orientadas a documentos pueden realizar *sharding* a través de técnicas de *hash* o según el valor de ciertos atributos.
- Las BD orientadas a columnas pueden usar tanto técnicas de fragmentación horizontal como vertical.

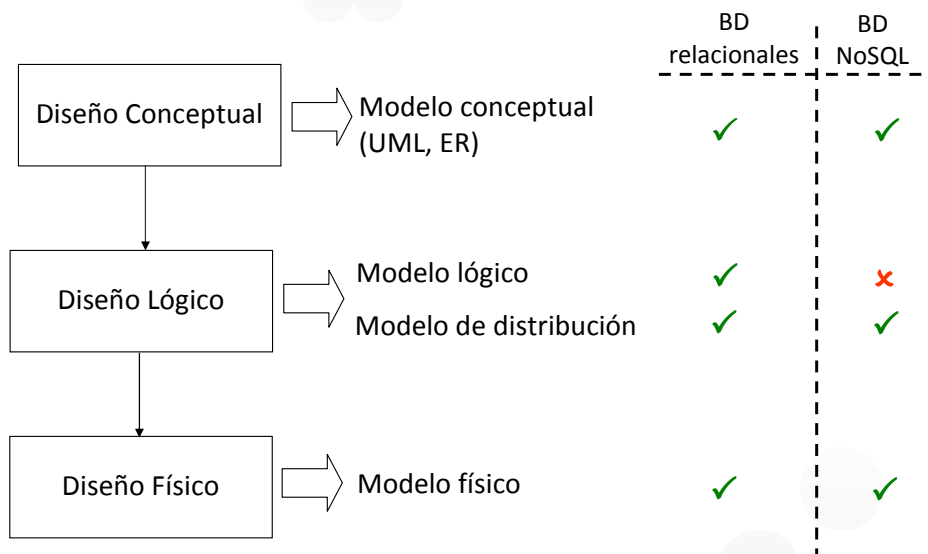
EIMT.UOC.EDU

En las bases de datos clave-valor la distribución de los agregados se hace a partir de su clave. Para ello, en algunos casos, se usan técnicas de *hash* (por ejemplo, *consistent hashing*) que de manera aleatoria y uniforme distribuyen los agregados entre los diferentes nodos. Por lo tanto, no hay garantía de que agregados relacionados estén en un mismo nodo. Es más, en algunos casos, no existen fragmentos (*shards*). En otras palabras, la unidad de distribución puede ser el agregado y no conjuntos de agregados. Es por ello que algunos fabricantes (por ejemplo, Riak) afirman que no usan *sharding*, sino únicamente replicación como estrategia de distribución.

En el caso de bases de datos orientadas a documentos, el *sharding* se puede efectuar, bien aplicando técnicas de *hash*, bien a partir del valor que toman ciertos atributos que siempre están presentes en el agregado (o documento). Un ejemplo de base de datos que admite ambas posibilidades es MongoDB. En MongoDB, los documentos de un mismo tipo se agrupan en colecciones y la fragmentación (*sharding*) se efectúa a nivel de colección. Las colecciones se pueden fragmentar según el valor de ciertos atributos, dando lugar a fragmentos (*shards*) que constituyen la unidad de distribución. Fijémonos que esta estrategia ayuda a resolver consultas por rangos que impliquen a los atributos usados en la fragmentación, dado que documentos con valores cercanos para esos atributos estarán posiblemente en el mismo fragmento. Los fragmentos son distribuidos automáticamente y de forma equitativa por el gestor de la base de datos entre los nodos. Si en algún momento la carga de trabajo se desequilibra (por ejemplo, un nodo pasa a almacenar un volumen de documentos muy superior al resto), el gestor la reequilibra de forma automática.

Las bases de datos orientadas a columnas también realizan el *sharding* a partir del valor que toma la clave. Recordemos que este modelo se puede ver como una tabla compuesta por filas. Cada fila queda identificada por la clave y representa un agregado. A su vez, cada agregado se puede organizar en columnas y/o familias de columnas. Si se desea que agregados con valores próximos para la clave queden agrupados en un mismo fragmento que, a su vez, se almacena en al menos un nodo, la estrategia seguida para generar las claves es de importancia primordial. A modo de ejemplo, BigTable utiliza URL invertidas como claves (por ejemplo, una URL como `www.cnn.com` se transformaría en `com.cnn.www`). Las claves se disponen en orden lexicográfico y se agrupan en rangos (cada rango recibe el nombre de *tablet*). La *tablet* (que vendría a ser un *shard*) es la unidad de distribución. Con esta estrategia, por ejemplo, se consigue que el contenido de páginas web de un mismo dominio se realice de forma eficiente dado que pertenecerán a una misma *tablet* y estarán, en consecuencia, almacenadas en un mismo nodo. Además, a nivel de cada nodo, BigTable permite que las columnas y familias de columnas de los agregados se puedan segregar en grupos (denominados *locality groups*), que a su vez, se almacenan localmente de forma separada. En definitiva, permite efectuar una fragmentación vertical. La segregación de columnas (o familias de columnas) que no se acceden de manera conjunta en grupos separados permite mejorar el ratio de datos útiles recuperados en las operaciones de consulta (sólo se leen los grupos de columnas o familias de columnas relevantes).

Consideraciones finales



EIMT.UOC.EDU

Para finalizar esta presentación queremos repasar brevemente las etapas en las que clásicamente se desglosa el diseño de bases de datos.

En la primera etapa se elabora un modelo conceptual de la base de datos, es decir, se obtiene una conceptualización de aquella parte del mundo real que nos interesa representar. El modelo conceptual es independiente de la tecnología de implementación, y para su elaboración podemos utilizar, por ejemplo, diagramas de clases de UML o el modelo ER.

Por su parte, en la segunda etapa, se procede a la transformación del modelo conceptual, dando como resultado un modelo lógico de la base de datos. Para la transformación, es necesario considerar el modelo de datos en el que se basa el sistema gestor de la base de datos que se usará. Así pues, el modelo lógico resultante será diferente si se trata de una base de datos relacional, una base de datos basada en un modelo clave-valor, documental, orientado a columnas o en grafo. Si la base de datos va a ser distribuida, en esta etapa, también será necesario diseñar el modelo de distribución. Las estrategias que se pueden usar para su elaboración han sido el foco de interés de esta presentación.

Finalmente, en la tercera etapa, es cuando se procede propiamente a la creación de la base de datos. Además de crear estructuras (sea de forma explícita o implícita) e insertar datos, se pueden tomar otro tipo de decisiones, por ejemplo, qué índices crear, dónde y cómo se almacenarán los datos localmente (en qué ficheros y dispositivos de almacenamiento) etc.

Los diferentes métodos que se pueden seguir en la primera etapa están bien estudiados y proceden del ámbito de la ingeniería del software. Por su parte, los métodos y técnicas a aplicar en las dos siguientes etapas pertenecen principalmente al ámbito de bases de datos. En el caso de bases de datos relacionales existe una teoría sólida que permite sistematizar el proceso de diseño lógico y físico. En el caso de las bases de datos NoSQL, está bien estudiado (tanto desde un punto de vista teórico como práctico) la elaboración del modelo de distribución y el diseño físico. Sin embargo, hay una cierta ausencia de teoría que indique cuáles serían las técnicas de modelado de datos más apropiadas para la elaboración del modelo lógico. Durante el curso hemos dado algunas indicaciones a este respecto. Aquellos de vosotros que queráis profundizar en estos aspectos, os recomendamos la cuarta de las referencias incluidas al final de esta presentación.

Bases de datos distribuidas

- ¿En qué consiste el diseño de BD distribuidas?
- Estrategias de distribución
- Diseño de BD distribuidas

En esta presentación hemos estudiado conceptos fundamentales asociados al diseño de bases de datos distribuidas. Entre ellos, destacan las estrategias de fragmentación y replicación que se pueden usar, con sus correspondientes adaptaciones, tanto en bases de datos relacionales como en bases de datos NoSQL.

Referencias

F. Chang et al. (2006). Bigtable: A Distributed Storage System for Structured Data, "Proceedings of the 7th Symposium on Operating Systems Design and Implementation".
(<http://static.googleusercontent.com/media/research.google.com/es//archive/bigtable-osdi06.pdf>)

C. Coronel, S. Morris & P. Rob (2013). *Database Systems: Design, Implementation and Management 10e*. Course Technology, Cengage Learning.

L. Liu & M.T. Özsu (Eds.) (2009). *Encyclopedia of Database Systems*. Springer.

I. Katsov (2012). "NoSQL Data Modeling Techniques", *Highly Scalable Blog. Articles on Big Data, NoSQL, and Highly Scalable Software Engineering*. (<http://highlyscalable.wordpress.com/2012/03/01/nosql-data-modeling-techniques/>)

M.T. Özsu & P. Valduriez (2011). *Principles of Distributed Systems*. 3rd edition. Springer.

O. Romero & M. Oliva (2012). Distributed Databases. Material docente UOC, asignatura Arquitectura de bases de datos.

P.J. Sadalage & M. Fowler. (2013). *NoSQL Distilled. A brief Guide to the Emerging World of Polyglot Persistence*, Pearson Education.

EIMT.UOC.EDU

Esperamos que hayáis disfrutado y aprendido con este vídeo. A continuación encontraréis algunas referencias que os permitirán profundizar más en los conceptos que hemos tratado.

Que tengáis un buen día.